Cluster and Cloud Computing Assignment 1

Assignment 1 Report

Group Member 1

Name: Xinyu Zhou

Student ID: 1434696

Email: xinyu.zhou10@student.unimelb.edu.au

Group Member 2

Name: JUNLANG HU

Student ID: 1461262

Email: junlang.hu@student.unimelb.edu.au

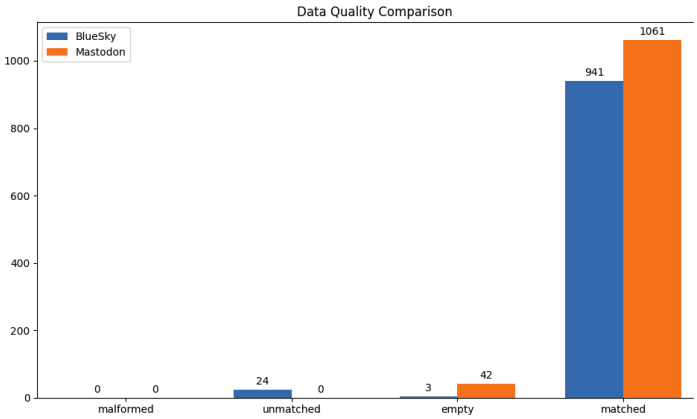
[This is the cover page, please do not count as report page (:] Reported: 11/04/2026

Introduction

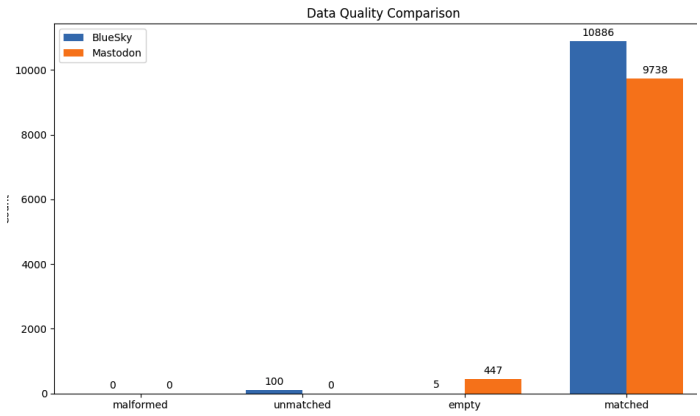
This project develops an MPI-based parallel application to analyse large-scale NDJSON datasets (Mastodon and BlueSky) on the Spartan HPC cluster. The program robustly extracts and aggregates ISO-639 language attributes. To evaluate scalability and I/O performance,the three parallelisation strategies—linear modulo, chunking, and worker-pool—are implemented and benchmarked across various node and core configurations. Performance trade-offs are subsequently analyzed by the Amdahl's Law.

Metadata Language Extracting and Normalisation

Before implementing the final parser, sample records from the small and medium datasets were examined to understand the structure of the NDJSON files. Preliminary inspection of the datasets revealed structural inconsistencies: BlueSky records store language attributes under `record.langs`, whereas Mastodon uses `doc.language`. Furthermore, extracted values varied in data types (strings versus lists). Edge cases, including malformed JSON objects (triggering `JSONDecodeError`), `unmatched` (No lang path) or `empty` (missing attributes), were also identified.



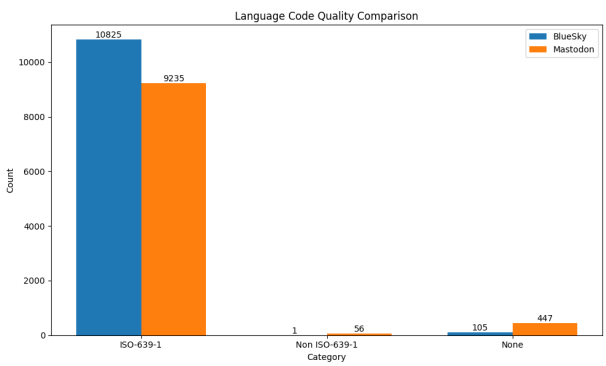
[Small Dataset Inspection]



[Medium Dataset Inspection]

To address these challenges, Each record is decoded via `json.loads` and queried by known schema paths. Extracted attributes are strictly converted into lists to ensure consistency. Invalid or empty records are safely bypassed. To preserve data fidelity and avoid unnecessary computational overhead, language identification relies exclusively on the provided metadata rather than natural language inference models (e.g., NLTK).

Also noticed in the inspection of metadata, some non structured languages, for example: `'zh': 207, 'ja': 81, 'zh-CN': 50` exist. To strictly adhere to ISO-639 standards, a normalisation layer was introduced to map tags to their primary language codes (e.g. aggregating `zh-CN` and `zh-`



`TW` into `zh`, or `en-US` into `en`). This guarantees an accurate and standardised distribution.

Parallelisation Strategies

The application utilizes a data-parallel architecture implemented via `mpi4py`. Processes within the global communicator (`MPI_COMM_WORLD`) independently compute local language frequencies. These local counters are subsequently aggregated at the root process (rank 0) via collective operations (`MPI_Reduce`). Execution boundaries are strictly managed using `MPI_Barrier`, and performance is tracked via `MPI_Wtime`.

To evaluate scalability, three distinct workload distribution strategies were developed, demonstrating key trade-offs between I/O efficiency, load balancing, and communication overhead. In the subsequent analysis, Amdahl's Law— $S(N) = \frac{T}{(1-\alpha)T + \alpha(T/N)}$ —is applied, where α represents the parallelisable workload fraction, $1-\alpha$ denotes the serial bottleneck, and T is the execution time on a single processor.

Method 1: Linear Modulo

In the linear modulo strategy, workload is statically distributed based on the line index. Each MPI process sequentially reads the entire input file but only executes the JSON parsing and metadata extraction on lines satisfying the condition: `line_number mod N = rank` (where N is the total number of processes).

Amdahl's Law Evaluation: From the perspective of Amdahl's Law, this strategy inherently limits the theoretical speedup $S(N)$. While the computational workload (JSON decoding and language counting) is effectively parallelised and reduced to $\alpha(T/N)$, the file I/O operation is entirely duplicated. Every process must read the entire dataset sequentially, which heavily inflates the non-parallelisable component $(1-\alpha)T$. Consequently, the file reading time cannot be reduced regardless of how many cores are added, creating a severe I/O bottleneck. Therefore, this method serves strictly as a functional baseline for comparison.

Method 2: Chunk

Unlike the linear modulo approach, the chunking strategy partitions the input file into contiguous byte segments of size `file_size / N`. Each MPI process utilizes `file.seek()` to jump directly to its assigned byte offset, reading and processing only its designated portion. To maintain data integrity across record boundaries (where a single JSON line may be split between chunks), each process (except rank 0) is programmed to skip its first partial line and instead process the subsequent full record. Crucially, the `json.read()` logic ensures the final record within a chunk is captured in its entirety; the process continues reading until encounter a newline character, even if this data exceeds the pre-calculated byte limit. This mechanism ensures that every record is processed exactly once without loss or duplication.

Amdahl's Law Evaluation: This strategy significantly optimizes execution time by distributing the I/O workload. In Method 1, the entire file-reading process was a redundant serial overhead. In Chunking, this I/O operation is successfully parallelised, shifting a substantial portion of the workload from the serial component $(1-\alpha)T$ into the parallelisable component.

However, perfect scalability remains constrained. The boundary handling logic and potential "data skew"—where some chunks contain more complex or numerous records than others—contribute to the non-parallelisable portion $(1-\alpha)T$. These factors, along with the synchronization required at the final aggregation stage, define the theoretical speedup limit.

Method 3: Worker Pool

Departing from static pre-partitioning, the worker-pool strategy adopts a "master–worker" model in which a single coordinator process (the master) actively manages the workload, dispatching predefined batches of data (e.g., groups of lines) to available worker processes on demand. Upon completing a task, a worker transmits its local result and immediately requests the next batch via MPI send/receive protocols, ensuring continuous resource utilization.

Amdahl's Law Evaluation: The primary advantage of this strategy is dynamic load balancing. By continuously feeding idle workers, it effectively mitigates the "data skew" problem inherent in Chunking, optimally minimizing the parallel execution time $\alpha(T/N)$. However, this structural advantage comes at a severe cost: continuous inter-process communication (handshakes and data payload transfers).

In the context of Amdahl's Law, this network overhead inflates the non-parallelisable serial bottleneck $(1-\alpha)T$. As the number of processes N scales—particularly across multiple physical nodes where network latency replaces shared-memory speed—this communication penalty often overrides the computational benefits of dynamic load balancing, limiting the overall speedup $S(N)$ for high-throughput, I/O-bound tasks.

Strategy Benchmarking and Selection (Medium Dataset)

	1n1c	1n8c	2n8c
Linear	0.3922	0.2516	0.2224
Chunk	0.3602	0.0571	0.0832
Work Pool	null	0.0832	0.1197

Before executing the final extraction on the 100GB large datasets, a benchmarking phase was conducted using medium-sized datasets to select the optimal parallelisation strategy. As demonstrated in the time comparison table, the Linear Modulo strategy exhibited poor

scalability due to redundant full-file I/O operations. The Worker-Pool approach performed adequately on 8 cores but encountered an inherent architectural limitation at 1n1c (null), as the Master-Worker topology strictly requires $N \geq 2$ processes to avoid deadlocks.

More importantly, Worker-Pool ultimately underperformed compared to Chunking (0.0832s vs. 0.0571s at 1n8c). This performance gap is directly attributed to the high uniformity of the NDJSON dataset. Because the social media records are relatively consistent in size and parsing complexity, the inherent "data skew" is minimal, rendering the dynamic load balancing of Worker-Pool computationally unnecessary. Instead, the continuous inter-process communication (frequent handshakes and task requests) inflated the serial bottleneck $(1-\alpha)T$.

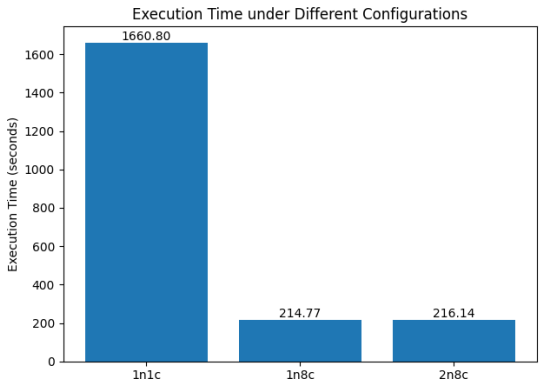
Ultimately, the Chunking strategy demonstrated the most superior performance. By achieving the highest I/O throughput and eliminating fine-grained communication penalties, Chunking was confidently selected as the definitive method for the large dataset execution.

Result and Discussion

The Result for large file language parsing

Mastodon Languages Used	Freq (#posts)	BlueSky Languages Used	Freq (#posts)
en	22,522,333	en	72,901,920
de	1,635,575	de	2,103,623
es	385,703	es	635,928
fr	383,728	fr	482,991
zh	328,864	pt	269,609
nl	221,422	pl	228,583
ru	189,917	ja	227,968
ja	178,651	fi	143,567
it	157,979	nl	109,264
fi	131,992	it	80,743

Final Large file executing



The execution times across different configurations perfectly illustrate the theoretical constraints defined by Amdahl' s Law: $S(N) = \frac{T}{(1-\alpha)T + \alpha(T/N)}$. When scaling from 1 core to 8 cores within a single node (1n8c), the execution time plummeted from 1660.80s to 214.77s, yielding an exceptional empirical speedup of $S(8) \approx 7.73$. Such near-linear scalability proves that the Chunking algorithm successfully converted the massive file I/O operations into a highly parallelisable workload $\alpha(T/N)$, leaving only a minimal serial fraction $(1-\alpha)T$.

However, further increasing from 1 node to 2 nodes does not result in additional performance improvement, the execution time slightly regressed to 216.14s. This efficiency drop occurs because transitioning to 2n8c shifts the `MPI_Reduce` and synchronization operations from ultra-fast intra-node shared memory to the inter-node physical network switch. This network latency fundamentally inflates the non-parallelisable communication overhead $(1-\alpha)T$. As dictated by Amdahl's Law, no matter how efficiently the data is chunked, this serial communication bottleneck ultimately limits infinite scalability.

Appendix LLM Declaration

This project received minor assistance from large language models for both implementation support and report writing. The use was only limited to research and information gathering. Code implementation and report construction and analysis were done on our own.

For the coding component, the LLM was used to assist understanding of MPI and components(methods, logic) about mpi4py. All core logic, including the design of parallelisation strategies , was developed independently.

For the report part, the LLM was used to refine wording and language choice. Some interaction involved discussing how to describe parallelisation trade-offs and Amdahl’ s Law. The final content reflects the our own understanding and interpretation.

In general, the LLM was used as a support tool for better understanding and studying of the project.

Involved LLM: Gemini, Chatgpt.